

Analyze runtime performance



Kayce Basques



(<https://twitter.com/kaycebasques>)



(<https://github.com/kaycebasques>)

Note: See [Optimize Website Speed \(/docs/devtools/lighthouse\)](/docs/devtools/lighthouse) to learn how make your pages load faster.

Runtime performance is how your page performs when it is running, as opposed to loading. This tutorial teaches you how to use the Chrome DevTools Performance panel to analyze runtime performance. In terms of the [RAIL](https://web.dev/rail/) (<https://web.dev/rail/>) model, the skills you learn in this tutorial are useful for analyzing the Response, Animation, and Idle phases of your page.

Caution: This tutorial is based on Chrome 129. If you use another version of Chrome, the UI and features of DevTools may be different. Check `chrome://help` to see what version of Chrome you're running.

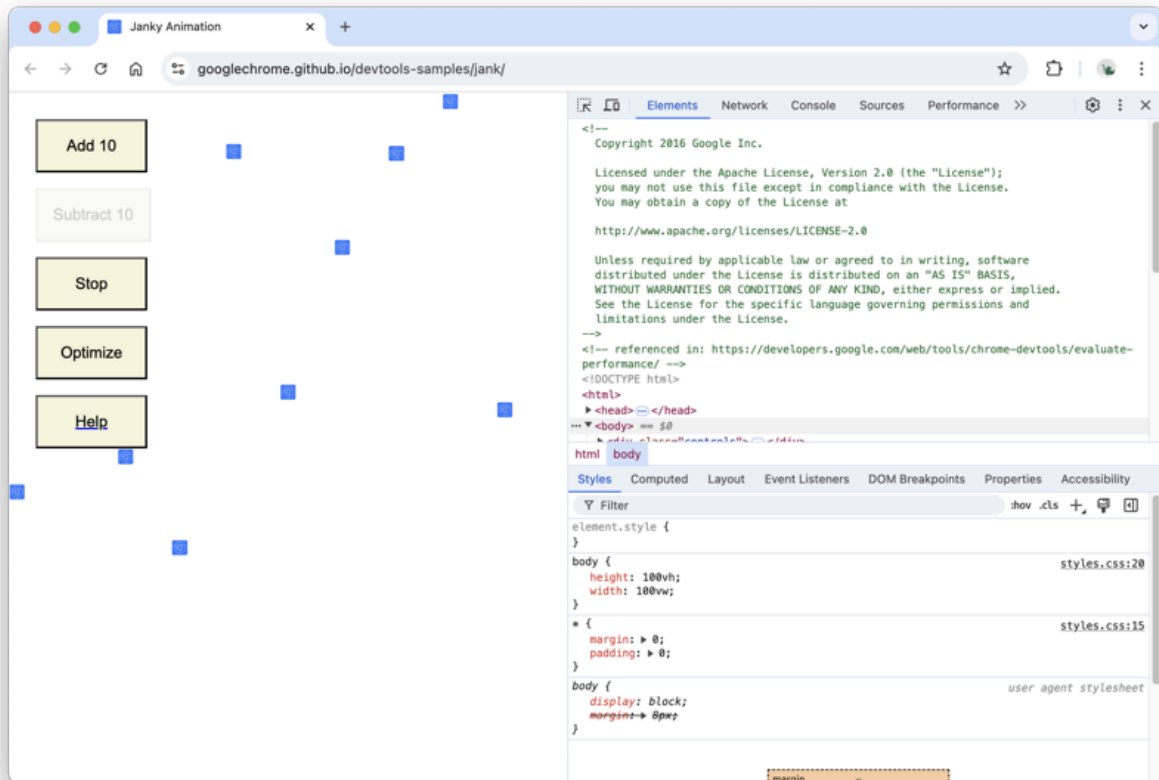
Get started

In this tutorial, we will use the **Performance** panel to find a performance bottleneck on a live page. To begin:

1. Open Google Chrome in [Incognito Mode](https://support.google.com/chrome/answer/95464) (<https://support.google.com/chrome/answer/95464>). Incognito Mode ensures that Chrome runs in a clean state. For example, if you have a lot of extensions installed, those extensions might create noise in your performance measurements.
2. Load the following page in your Incognito window. This is the demo that you're going to profile. The page shows a bunch of little blue squares moving up and down.

<https://googlechrome.github.io/devtools-samples/jank/>

3. Press Command+Option+I (Mac) or Control+Shift+I (Windows, Linux) to open DevTools.



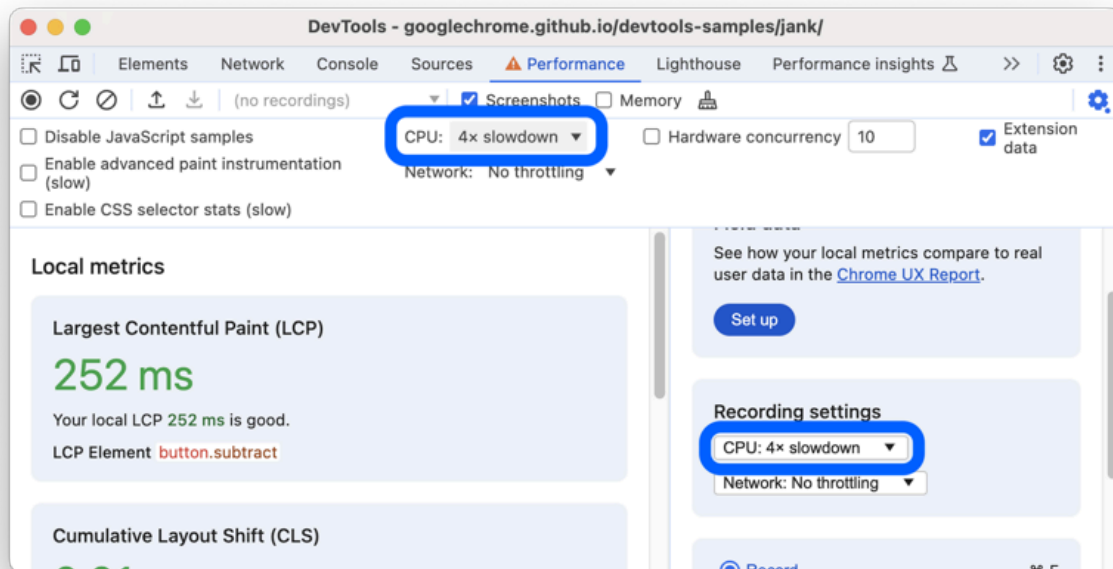
★ **Note:** For the rest of the screenshots, DevTools is undocked to a separate window (<https://developer.mozilla.org/docs/Web/API/window/requestAnimationFrame>) so that you can see its contents better.

Simulate a mobile CPU

Mobile devices have much less CPU power than desktops and laptops. Whenever you profile a page, use CPU Throttling to simulate how your page performs on mobile devices.

1. In DevTools, click the **Performance** tab.
2. Make sure that the **Screenshots** ☒ checkbox is enabled.

3. Click **Capture Settings** ⚙️. DevTools reveals settings related to how it captures performance metrics.
4. For **CPU**, select **4x slowdown**. DevTools throttles your CPU so that it's 4 times slower than usual.



★ **Note:** When testing other pages, if you want to ensure that they work well on low-end mobile devices, set CPU Throttling to **20x slowdown**. This demo doesn't work well with 20x slowdown, so it just uses 4x slowdown for instructional purposes.

Set up the demo

It's hard to create a runtime performance demo that works consistently for all readers of this website. This section lets you customize the demo to ensure that your experience is relatively consistent with the screenshots and descriptions you see in this tutorial, regardless of your particular setup.

1. Keep clicking **Add 10** until the blue squares move noticeably slower than before. On a high-end machine, it may take about 20 clicks.

2. Click **Optimize**. The blue squares should move faster and more smoothly.

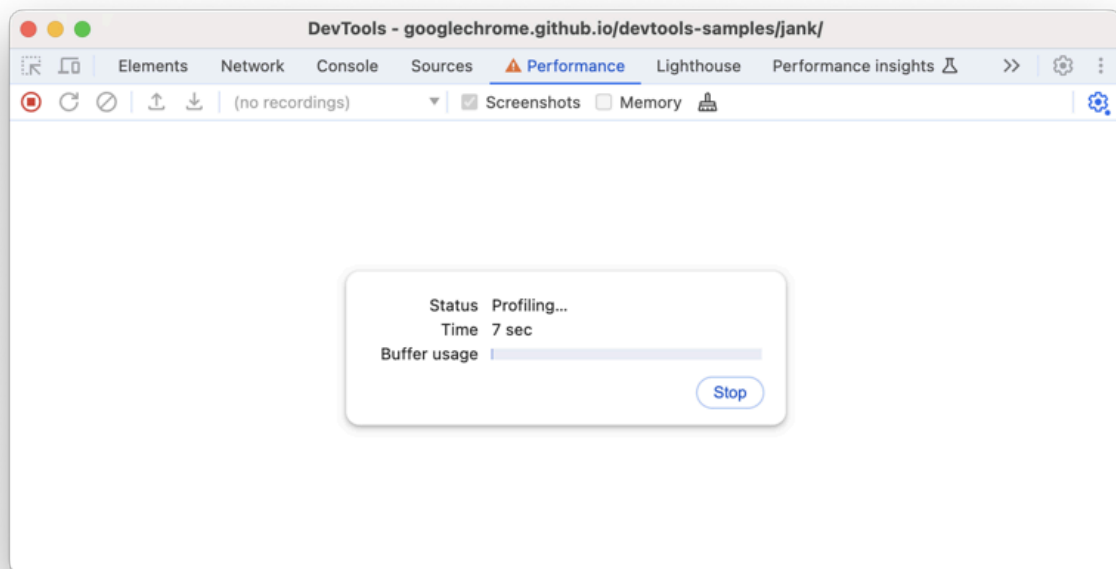
★ **Note:** If you don't see a noticeable difference between the optimized and un-optimized versions, try clicking **Subtract 10** a few times and trying again. If you add too many blue squares, you're just going to max out the CPU and you're not going to see a major difference in the results for the two versions.

3. Click **Un-Optimize**. The blue squares move slower and with more jank again.

Record runtime performance

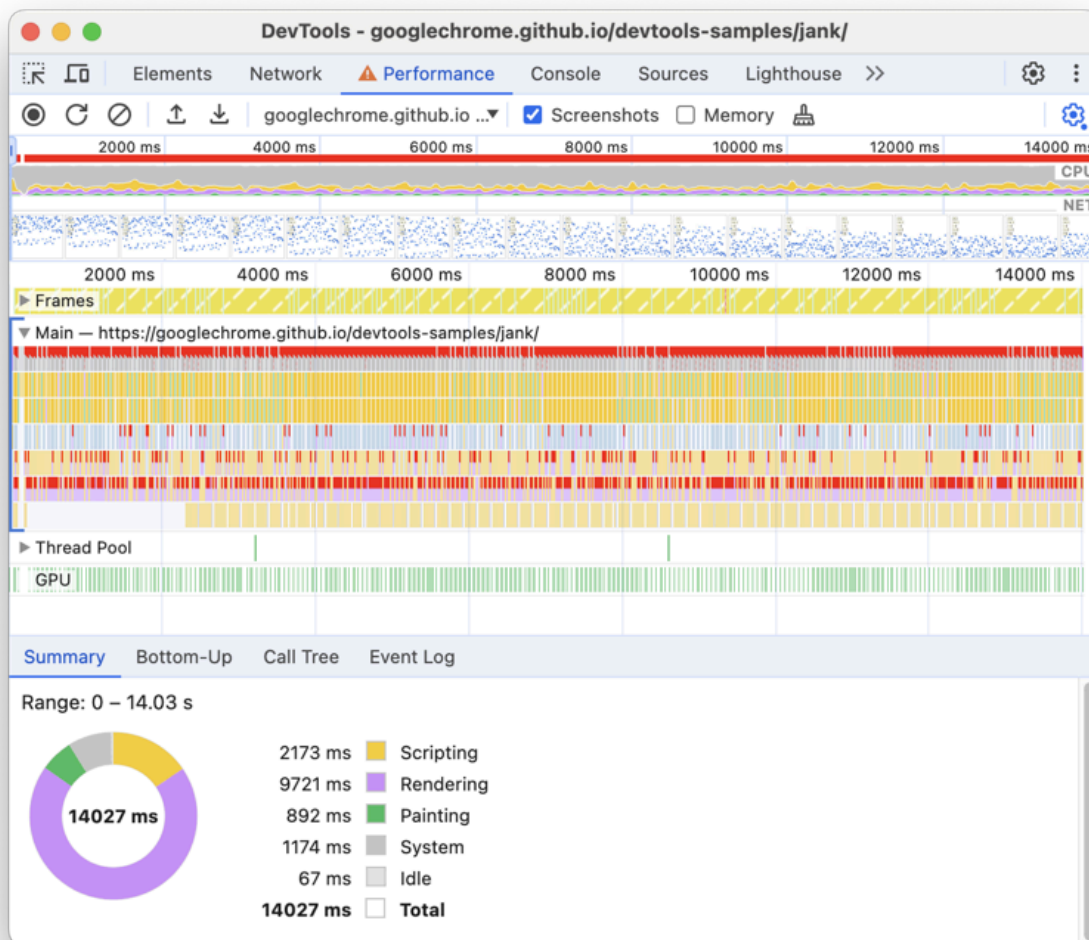
When you ran the optimized version of the page, the blue squares move faster. Why is that? Both versions are supposed to move each square the same amount of space in the same amount of time. Take a recording in the **Performance** panel to learn how to detect the performance bottleneck in the un-optimized version.

1. In DevTools, click **Record** . DevTools captures performance metrics as the page runs.



2. Wait a few seconds.

3. Click **Stop**. DevTools stops recording, processes the data, then displays the results in the **Performance** panel.



Wow, that's an overwhelming amount of data. Don't worry, it'll make more sense shortly.

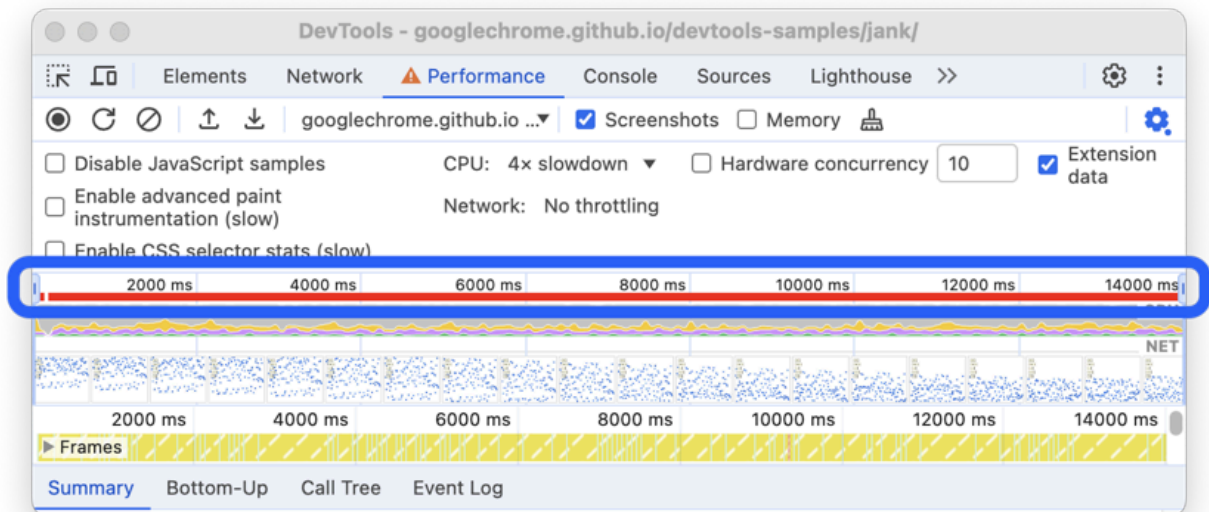
Analyze the results

Once you have a performance recording, you can analyze just how poor the page's performance is, and find the cause(s).

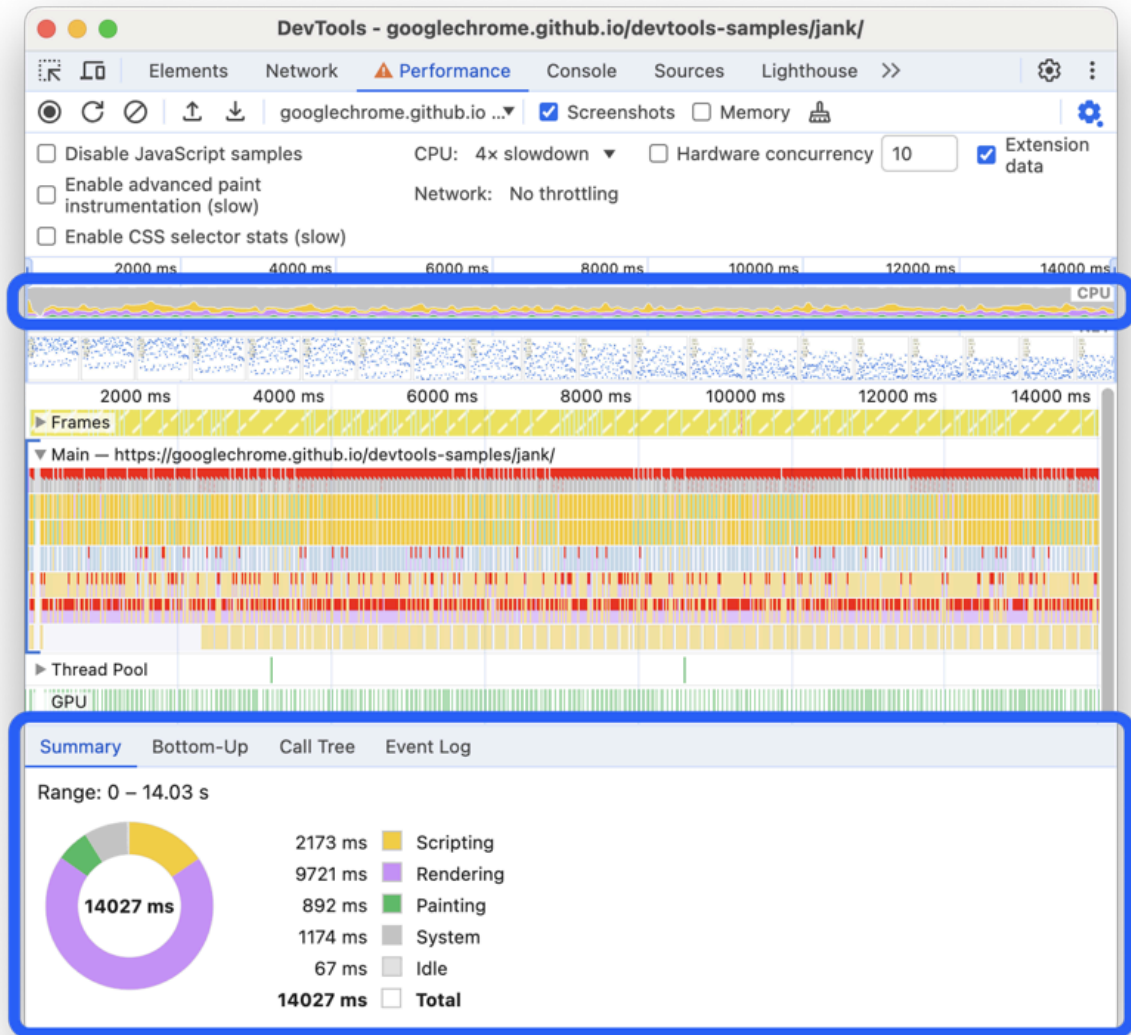
Analyze frames per second

The main metric for measuring the performance of any animation is frames per second (FPS). Users are happy when animations run at 60 FPS.

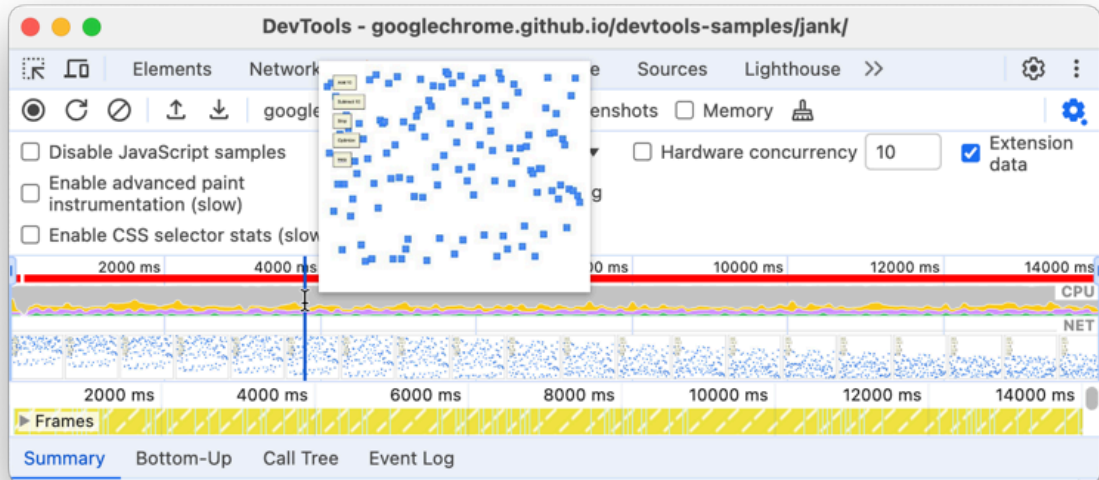
1. Look at the **FPS** chart. Whenever you see a red bar above **FPS**, it means that the framerate dropped so low that it's probably harming the user experience.



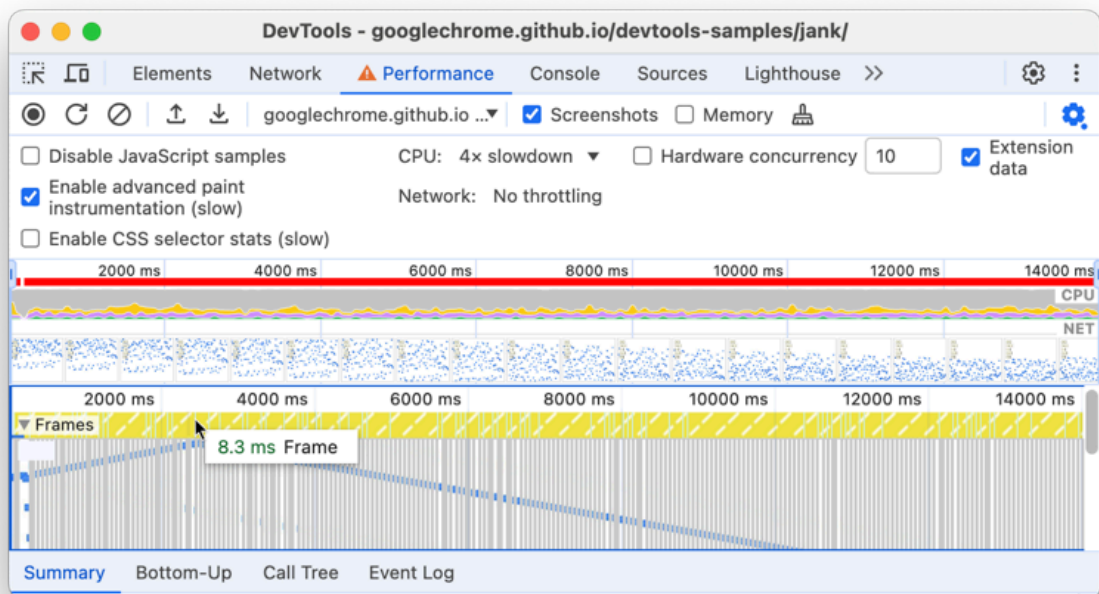
2. Below the **FPS** chart you see the **CPU** chart. The colors in the **CPU** chart correspond to the colors in the **Summary** tab, at the bottom of the **Performance** panel. The fact that the **CPU** chart is full of color means that the CPU was maxed out during the recording. Whenever you see the CPU maxed out for long periods, it's a cue to find ways to do less work.



3. Hover your mouse over the **FPS**, **CPU**, or **NET** charts. DevTools shows a screenshot of the page at that point in time. Move your mouse left and right to replay the recording. This is called scrubbing, and it's useful for manually analyzing the progression of animations.



4. In the **Frames** section, hover your mouse over one of the green squares. DevTools shows you the FPS for that particular frame. Each frame is probably well below the target of 60 FPS.



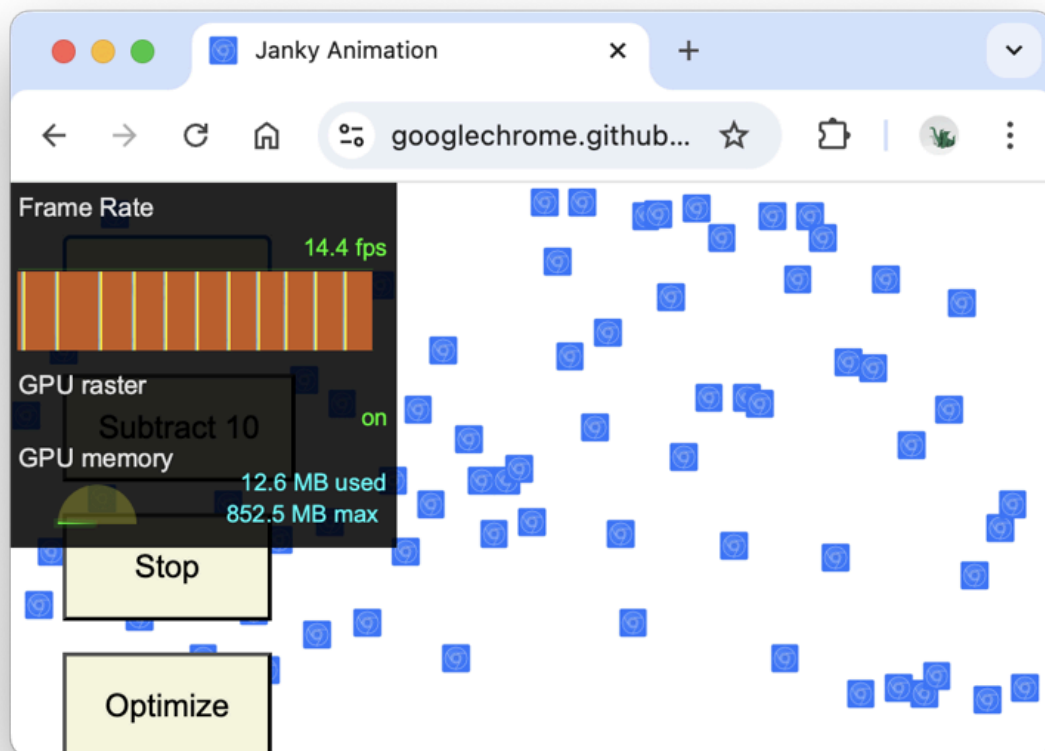
Of course, with this demo, it's pretty obvious that the page is not performing well. But in real scenarios, it may not be so clear, so having all of these tools to make measurements comes in

handy.

Bonus: Open the FPS meter

Another handy tool is the **FPS meter**, which provides real-time estimates for FPS as the page runs.

1. Press Command+Shift+P (Mac) or Control+Shift+P (Windows, Linux) to open the Command Menu.
2. Start typing **Rendering** in the Command Menu and select **Show Rendering**.
3. In the **Rendering** panel, enable **Show Rendering stats**. A new overlay appears in the top-right of your viewport.

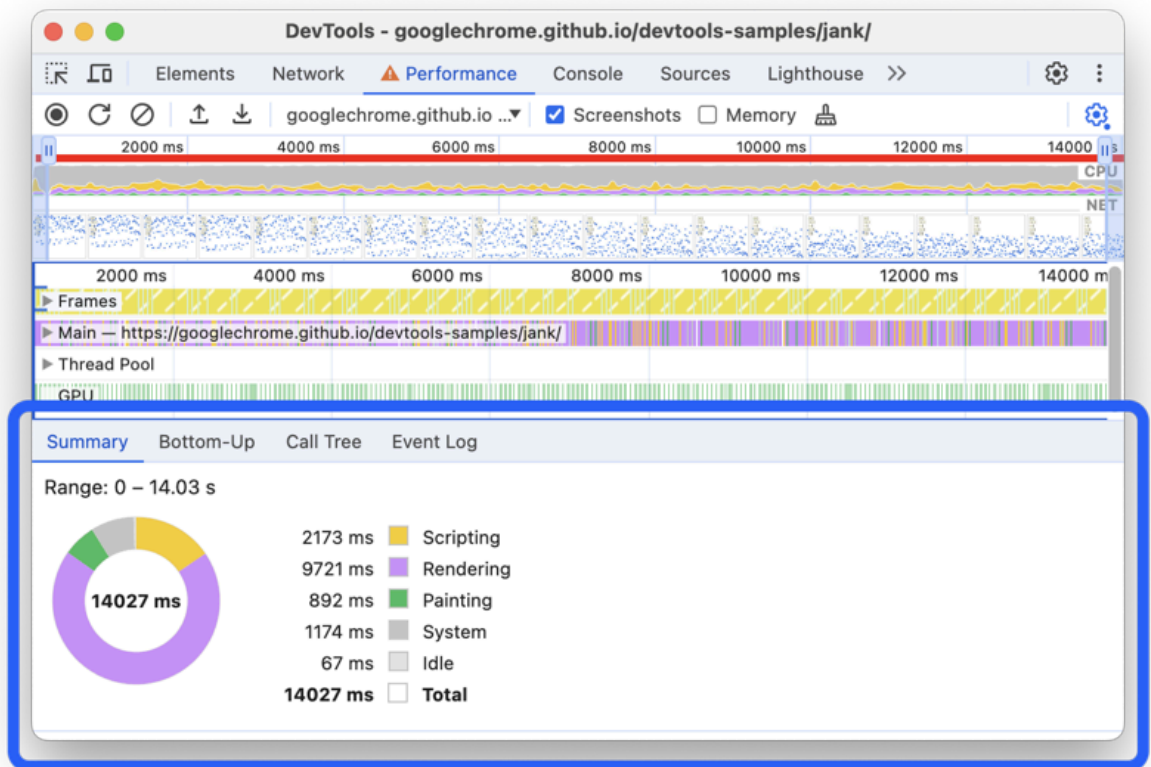


4. Disable the **FPS Meter** and press Escape to close the **Rendering** panel. You won't be using it in this tutorial.

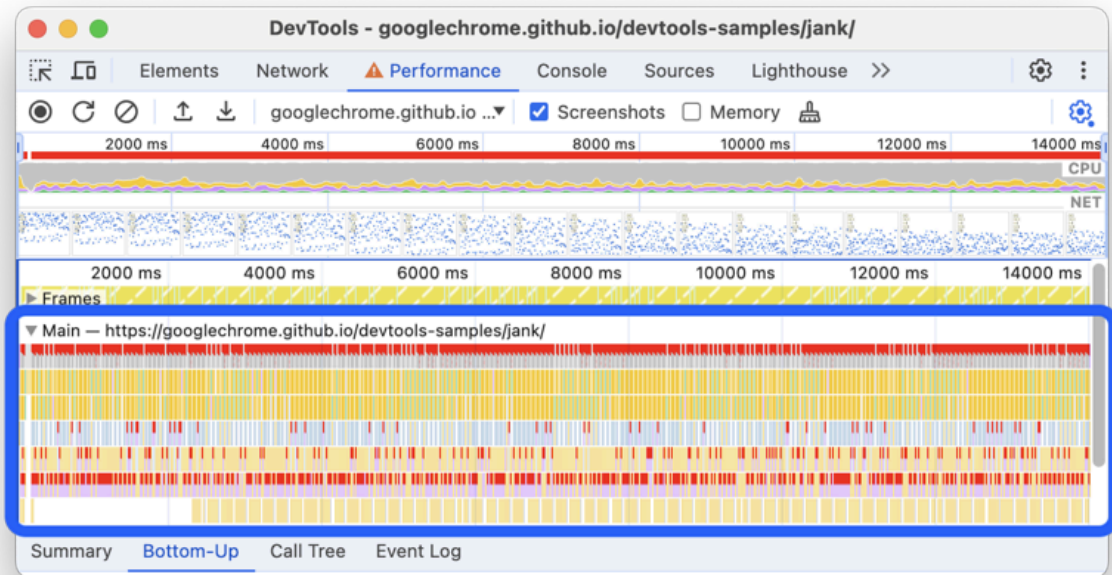
Find the bottleneck

Now that you've measured and verified that the animation is not performing well, the next question to answer is: why?

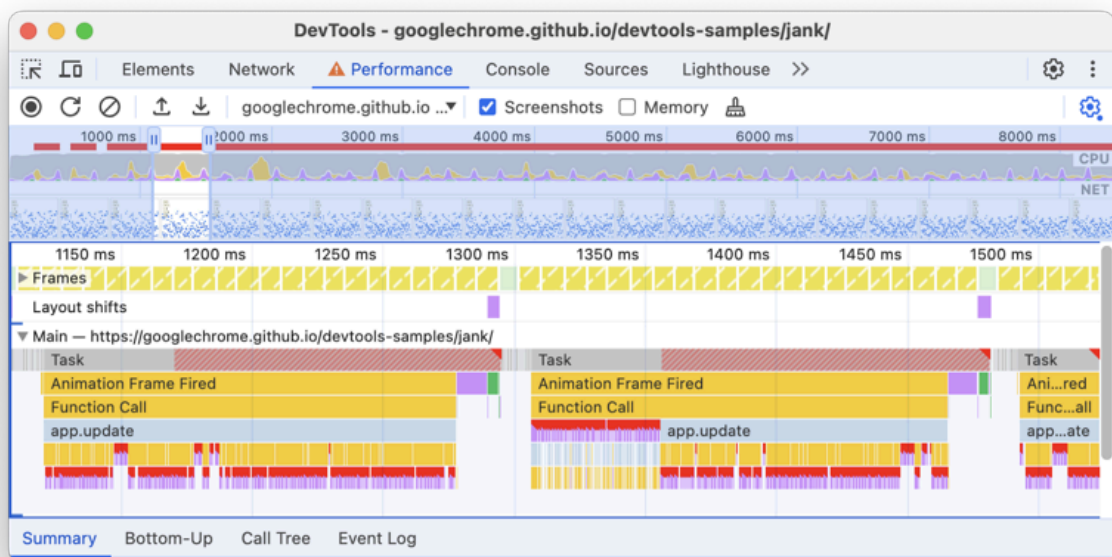
1. Note the **Summary** tab. When no events are selected, this tab shows you a breakdown of activity. The page spent most of its time rendering. Since performance is the art of doing less work, your goal is to reduce the amount of time spent doing rendering work.



2. Expand the **Main** section. DevTools shows you a flame chart of activity on the main thread, over time. The x-axis represents the recording, over time. Each bar represents an event. A wider bar means that event took longer. The y-axis represents the call stack. When you see events stacked on top of each other, it means the upper events caused the lower events.

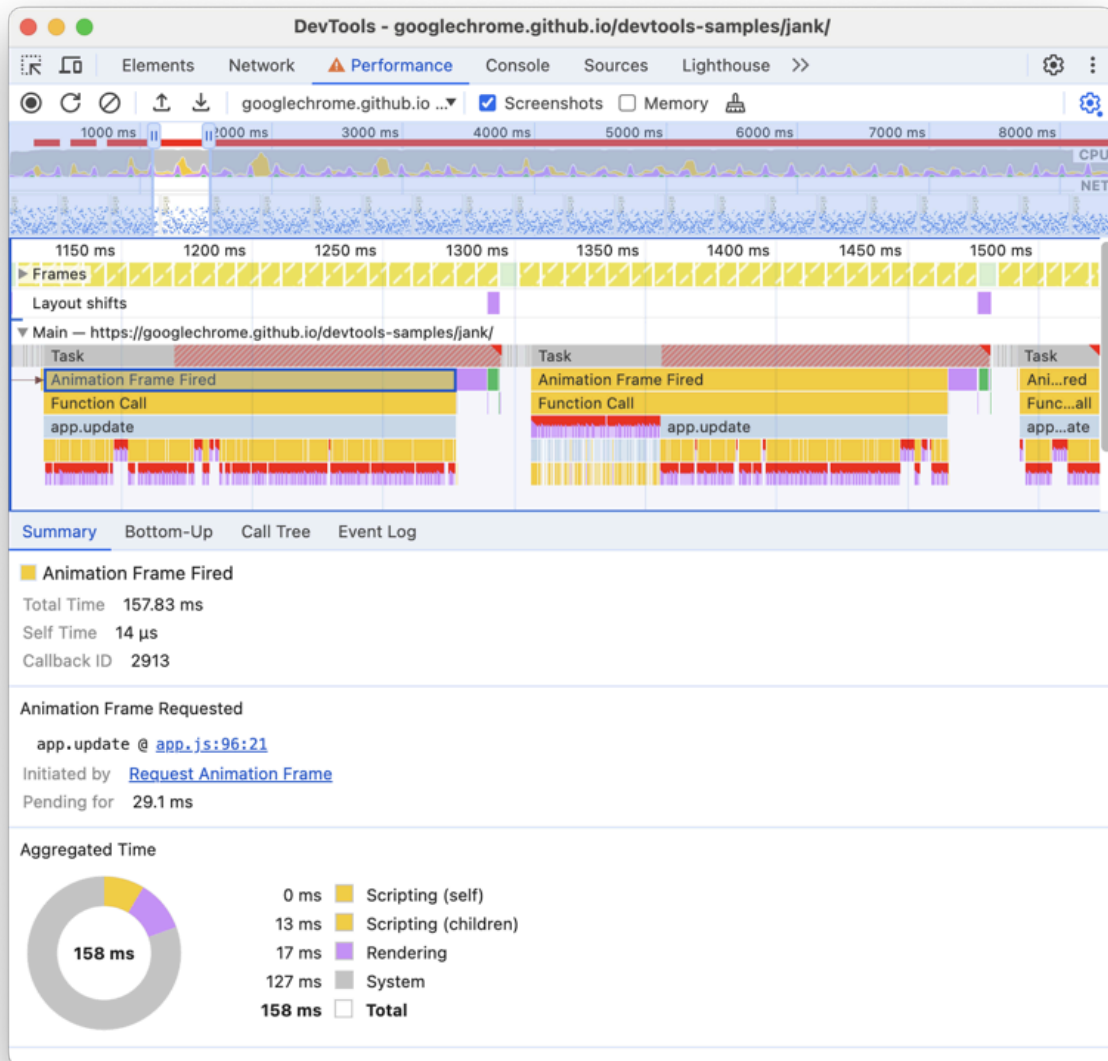


3. There's a lot of data in the recording. Zoom in on a single **Animation Frame Fired** event by clicking, holding, and dragging your mouse over the **Overview**, which is the section that includes the **FPS**, **CPU**, and **NET** charts. The **Main** section and **Summary** tab only display information for the selected portion of the recording.



★ **Tip:** You can also use WASD to navigate the recording. Focus the **Main** section by clicking its background or selecting an event, then press W to zoom in, S to zoom out, A to scroll left, or D to scroll right.

4. Note the red triangle in the top-right of the **Task** and layout events. Whenever you see a red triangle, it's a warning that there may be an issue related to this event. A red triangle on a **Task** means that it was a long task (https://web.dev/articles/optimize-long-tasks?utm_source=devtools).
5. Click the **Animation Frame Fired** event. The **Summary** tab now shows you information about that event. Clicking the link next to **Initiated by** causes DevTools to highlight the event that initiated the **Animation Frame Fired** event. Also note the **app.update @** link. Clicking that jumps you to the relevant line in the source code.



★ **Note:** After selecting an event, use the arrow keys to select the events next to it.

- Under the **app.update** event, there's a bunch of purple events. If they were wider, it looks as though each one might have a red triangle on it. Click one of the purple **Layout** events now. DevTools provides more information about the event in the **Summary** tab. Indeed, there's a warning about forced reflows (another word for layout).
- In the **Summary** tab, click the link next to **app.update @** under **Animation Frame Requested**. DevTools takes you to the line of code that forced the layout.



★ **Note:** The problem with this code is that, in each animation frame, it changes the style for each square, and then queries the position of each square on the page. Because the styles changed, the browser doesn't know if each square's position changed, so it has to re-layout the square in order to compute its position. See [Avoid forced synchronous layouts](https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing#avoid-forced-synchronous-layouts) (<https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing#avoid-forced-synchronous-layouts>) to learn more.

Phew! That was a lot to take in, but you now have a solid foundation in the basic workflow for analyzing runtime performance. Good job.

Bonus: Analyze the optimized version

Using the workflows and tools that you just learned, click **Optimize** on the demo to enable the optimized code, take another performance recording, and then analyze the results. From the improved framerate to the reduction in events in the **Main** section's flame chart, you can see that the optimized version of the app does much less work, resulting in better performance.

Note: Even this optimized version isn't that great, because it still manipulates the **top** property of each square. A better approach is to stick to properties that only affect compositing. See [Use transform and opacity changes for animations](#)

(<https://web.dev/articles/stick-to-compositor-only-properties-and-manage-layer-count#use-transform-and-opacity-changes-for-animations>)

for more information.

Next steps

The foundation for understanding performance is the RAIL model. This model teaches you the performance metrics that are most important to your users. See [Measure Performance With The RAIL Model](#) (<https://web.dev/rail/>) to learn more.

To get more comfortable with the Performance panel, practice makes perfect. Try profiling your own pages and analyzing the results. If you have any questions about your results, [open a Stack Overflow question tagged with google-chrome-devtools](#)

(<http://stackoverflow.com/questions/ask?tags=google-chrome-devtools>). Include screenshots or links to reproducible pages, if possible.

To become an expert in runtime performance, you've got to learn how the browser translates HTML, CSS, and JS into pixels on a screen. The best place to start is the [Rendering Performance Overview](#) (<https://web.dev/rendering-performance/>). [The Anatomy Of A Frame](#) (<https://aerotwist.com/blog/the-anatomy-of-a-frame/>) dives into even more detail.

Last, there are many ways to improve runtime performance. This tutorial focused on one particular animation bottleneck to give you a focused tour through the Performance panel, but it's only one of many bottlenecks you may encounter. The rest of the Rendering Performance series has a lot of good tips for improving various aspects of runtime performance, such as:

- [Optimizing JS Execution](#) (<https://web.dev/optimize-javascript-execution/>)
- [Reduce The Scope And Complexity Of Style Calculations](#)
(<https://web.dev/reduce-the-scope-and-complexity-of-style-calculations/>)

- Avoid Large, Complex Layouts And Layout Thrashing
(<https://web.dev/avoid-large-complex-layouts-and-layout-thrashing/>)
- Simplify Paint Complexity And Reduce Paint Areas
(<https://web.dev/simplify-paint-complexity-and-reduce-paint-areas/>)
- Stick To Compositor-Only Properties And Manage Layer Count
(<https://web.dev/stick-to-compositor-only-properties-and-manage-layer-count/>)
- Debounce Your Input Handlers (<https://web.dev/debounce-your-input-handlers/>)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-10-23 UTC.